

Spring 26 CSCI 240 -- PA 7

Hash Maps

Feel free to discuss and help each other out but does not imply that you can give away your code or your answers! Make sure to read all instructions before attempting this lab.

You can work with a lab partner and each one must submit the same PDF file (include both names in the submission file). Each person must include a brief statement about your contribution to this assignment.

You must use an appropriate provided template from Canvas and output "Author: Your Name(s)" for all your programs. If you are modifying an existing program, use "Modified by: Your Name(s)".

Exercise 1 – Use C++ language STL **unordered_map** or Java language **HashMap** to store the following integer keys: 13 21 5 37 15 (use the reverse of the key as a string for the value part so first entry would be <13, "31">). Perform the following operations to make sure it is working properly: search for 10 and 21, remove 20, 37, and then search for 37. If an item is found, print the key and its value. Use an iterator to print the entries in the map.

Input data file **small1k.txt**, containing a list of 1,000 integer values and insert all the pairs <int, reverse key as string> to a new hash map. Use table size of 1000 / 0.75 (load factor of about 75%). Collect the time it took to insert 1,000 pairs of values to the hash map and output the time to the screen.

Input data file **large100k.txt**, containing a list of 100,000 integer values and insert all the pairs <int, reverse key as string> to another new hash map. Use table size of 100000 / 0.75 (load factor of about 75%). Collect the time it took to insert 100,000 pairs of values to the hash map and output the time to the screen.

You should confirm that it should only take about 100 times slower to process the larger file comparing to the smaller file.

Exercise 2 – Use C++ **ChainHashMap** (Java book) or Java **ChainHashMap** (Java book) with $N = 11$ and test it out with the same data and test cases from exercise 1. Use an iterator to print the entries in the map. Collect the times for the 2 data files like exercise 1 and use table size of $N / 0.75$ (load factor of about 75%). They should be very similar to the times in exercise 1 and you should confirm that is the case.

Exercise 3 – Perform a comparative analysis that studies the collision rates for various hash codes for character strings, such as polynomial hash codes for different values of the parameter a or cyclic-shift hash codes with different values for the bit shift. Use a hash

table to determine collisions, but only count collisions where different strings map to the same hash code (not if they map to the same location in this hash table). Use 1 (not a good value), 37, 40, and 41 for parameter **a** with polynomial hash codes or a shift of 0 (not a good value), 1, 5, and 13 for cyclic shift hash codes (only need to pick one method, either polynomial hash codes or cyclic shift hash codes). Test it on the formatted text file, US Declaration of Independence, and you can assume that words are separated by white space characters (easier to work with the formatted file). Ignore words already seen so do not count them or generate hash code for repeated words again. Output number of unique words in the file and the number of collisions for each case. Make sure the results are reasonable (one of the 4 values should give a much higher number of collisions).

Question 1: Do collected times for exercises 1 and 2 make sense? Explain why or why not.

Question 2: Why do we need to use compression functions? What are some common compression functions?

You can earn EC for only one of the options below:

Extra Credit option 1 (2 points): Collect data for second hash code for exercise 3.

Extra Credit option 2 (more extra work, 3 points): Create a program to collect some data about chain hashing that you worked on in exercise 2. You would need to try the large input data file, *large100k.txt*, with different load factors. After all the entries are inserted to the table, print the **table size, average number of probes in floating-point format (at least 1.0), and maximum number of probes for the worst case**. It would take at least one probe for each insertion (checking initial location). Therefore, it would be two probes if a second location is examined. You must try the load factors of 0.25, 0.5, 0.75, and 0.9 against the large input file (4 different sets of output). Make sure that your hash table size is determined correctly according to the load factor by using the following formula: N / LF .

Fill out and turn in the PA submission file for this assignment (save as PDF format).